

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 12_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 45

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items such that their total weight is exactly equal to a given target weight. Each item has a known weight, and every item can be selected at most once.

Given a list of item weights and a target weight k , determine whether there exists a subset of items whose total weight is exactly k .

If such a subset exists, print any one valid subset.

If no such subset exists, print -1.

The solution must be implemented using Dynamic Programming.

Formula

Let

$dp[i][j] = \text{true}$

if a subset of the first i items can form a sum j .

Transition:

If $arr[i-1] > j$

$dp[i][j] = dp[i-1][j]$

Else

$dp[i][j] = dp[i-1][j] \text{ OR } dp[i-1][j - arr[i-1]]$

Input Format

The first line contains two integers n and k , where n is the number of items and k is the target weight.

The second line contains n space-separated integers representing item weights.

Output Format

If a valid subset exists, print the elements of one such subset in a single line separated by spaces.

If no subset exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 9
3 34 4 12 5
Output: 5 4

Answer

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, k;
    scanf("%d %d", &n, &k);

    int arr[105];
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bool dp[105][1005] = {false};

    for(int i = 0; i <= n; i++)
        dp[i][0] = true;

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= k; j++) {
            if(arr[i-1] > j)
                dp[i][j] = dp[i-1][j];
            else
                dp[i][j] = dp[i-1][j] || dp[i-1][j - arr[i-1]];
        }
    }

    if(!dp[n][k]) {
        printf("-1");
        return 0;
    }

    int i = n, j = k;
    int res[105], idx = 0;

    while(i > 0 && j > 0) {
        if(dp[i-1][j]) {
            i--;
        } else {
            res[idx++] = arr[i-1];
            j -= arr[i-1];
        }
    }

    for(int i = idx-1; i >= 0; i--)
        printf("%d ", res[i]);
    printf("\n");
}
```

```
        res[idx++] = arr[i-1];  
        j -= arr[i-1];  
        i--;  
    }  
}  
  
for(int x = 0; x < idx; x++)  
    printf("%d ", res[x]);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Ramya is working as a logistics coordinator for a delivery company. Every day, she receives a list of delivery locations arranged along a straight highway. Each location has a certain delivery priority value (which may be positive or negative depending on profit or loss).

Ramya wants to choose a continuous sequence of delivery locations such that the total delivery priority is maximized, ensuring maximum profit for that day.

Since the company handles a large number of deliveries, Ramya needs an efficient solution that runs in polynomial time.

Input Format

The first line contains an integer n , representing the number of delivery locations.

The second line contains n space-separated integers, representing the priority values.

Output Format

The output prints a single integer representing the maximum total priority of any contiguous sequence of locations.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8
-2 3 5 -1 4 -5 2 6

Output: 14

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);

    int x;
    scanf("%d", &x);

    int max_so_far = x, current = x;

    for(int i = 1; i < n; i++) {
        scanf("%d", &x);
        if(current + x > x)
            current = current + x;
        else
            current = x;

        if(current > max_so_far)
            max_so_far = current;
    }

    printf("%d", max_so_far);
    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragav is managing a chain of warehouses located along a long highway. Each warehouse is positioned at a specific coordinate on the road.

To improve efficiency, Ragav wants to choose a central hub location such that the total travel distance from all warehouses to this hub is minimized.

He can place the hub at any one of the existing warehouse locations.

Since the number of warehouses can be very large, Ragav needs an efficient algorithm that runs in polynomial time to compute the optimal location.

Input Format

The first line contains an integer n , representing the number of warehouses.

The second line contains n space-separated integers, representing the positions of warehouses on the highway.

Output Format

The output prints a single integer representing the minimum total distance from all warehouses to the chosen hub.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2 3 6 10

Output: 13

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int cmp(const void *a, const void *b) {
```

```
    long long x = *(long long*)a;
```

```
    long long y = *(long long*)b;
```

```
    if(x < y) return -1;
```

```

    if(x > y) return 1;
    return 0;
}

int main() {
    int n;
    scanf("%d", &n);

    long long arr[100005];
    for(int i = 0; i < n; i++)
        scanf("%lld", &arr[i]);

    qsort(arr, n, sizeof(long long), cmp);

    long long median = arr[n/2];
    long long sum = 0;

    for(int i = 0; i < n; i++) {
        long long diff = arr[i] - median;
        if(diff < 0) diff = -diff;
        sum += diff;
    }

    printf("%lld", sum);
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Peter is a project manager in a software company. He has a list of employees, and each employee has a certain skill contribution value (which can be positive or negative depending on their impact on the project).

Peter wants to form a team such that the total skill contribution is exactly equal to a given target value. Each employee can be selected at most once, and the team must contain at least one employee.

Since the number of possible teams grows rapidly with the number of employees, Peter needs to determine whether such a team exists.

Input Format

The first line contains an integer n , representing the number of employees.

The second line contains n space-separated integers, representing the skill contribution of each employee.

The third line contains an integer $target$, representing the required total skill.

Output Format

If a valid team exists, the output prints: "Team Found: x_1 x_2 x_3 ..."

If no such team exists, the output prints: "No valid team found"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

3 7 -2 5 1

6

Output: Team Found: 3 -2 5

Answer

```
#include <stdio.h>
```

```
int n, target;
```

```
int arr[25], res[25];
```

```
int solve(int i, int sum, int idx) {
```

```
    if(i == n) {
```

```
        if(sum == target && idx > 0) {
```

```
            printf("Team Found: ");
```

```
            for(int j = 0; j < idx; j++)
```

```
                printf("%d ", res[j]);
```

```
            return 1;
```

```
        }
```



```

        return 0;
    }

    res[idx] = arr[i];
    if(solve(i + 1, sum + arr[i], idx + 1))
        return 1;

    if(solve(i + 1, sum, idx))
        return 1;

    return 0;
}

int main() {
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    scanf("%d", &target);

    if(!solve(0, 0, 0))
        printf("No valid team found");

    return 0;
}

```

Status : Partially correct

Marks : 5/10

5. Problem Statement

Viyana is an event coordinator organizing multiple sessions for a tech conference. Each session has a start time and an end time.

Due to limited resources, Viyana wants to select a group of sessions such that:

No two selected sessions overlap in time. The total number of selected sessions is maximized.

However, to make the event more impactful, each session also has a priority score, and Viyana wants to maximize the total priority score instead of just the count.

Your task is to help Viyana select the optimal set of non-overlapping sessions that yields the maximum total priority score.

Input Format

The first line contains an integer n , representing the number of sessions.

The next n lines each contain three integers: start time, end time and priority score.

Output Format

The first line of output prints the maximum total priority score.

In the next line, print the selected sessions (indices starting from 1) in ascending order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 3 5

2 5 6

4 6 5

6 7 4

Output: 14

1 3 4

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int s, e, w, idx;  
} Node;
```

```

int cmp(const void *a, const void *b) {
    Node *x = (Node*)a;
    Node *y = (Node*)b;
    if(x->e != y->e) return x->e - y->e;
    return x->s - y->s;
}

```

```

int main() {
    int n;
    scanf("%d", &n);

    Node arr[25];
    for(int i = 0; i < n; i++) {
        scanf("%d %d %d", &arr[i].s, &arr[i].e, &arr[i].w);
        arr[i].idx = i + 1;
    }

```

```

    qsort(arr, n, sizeof(Node), cmp);

```

```

    int p[25];
    for(int i = 0; i < n; i++) {
        p[i] = -1;
        for(int j = i - 1; j >= 0; j--) {
            if(arr[j].e <= arr[i].s) {
                p[i] = j;
                break;
            }
        }
    }
}

```

```

    int dp[25];
    dp[0] = arr[0].w;

```

```

    for(int i = 1; i < n; i++) {
        int incl = arr[i].w + (p[i] != -1 ? dp[p[i]] : 0);
        int excl = dp[i-1];
        dp[i] = incl > excl ? incl : excl;
    }

```

```

    printf("%d\n", dp[n-1]);

```

```

    int res[25], cnt = 0;

```

```

int i = n - 1;
while(i >= 0) {
    int incl = arr[i].w + (p[i] != -1 ? dp[p[i]] : 0);
    int excl = (i > 0) ? dp[i-1] : 0;

    if(incl >= excl) {
        res[cnt++] = arr[i].idx;
        i = p[i];
    } else {
        i--;
    }
}

for(int i = 0; i < cnt - 1; i++) {
    for(int j = i + 1; j < cnt; j++) {
        if(res[i] > res[j]) {
            int t = res[i];
            res[i] = res[j];
            res[j] = t;
        }
    }
}

for(int i = 0; i < cnt; i++)
    printf("%d ", res[i]);

return 0;
}

```

Status : Correct

Marks : 10/10